

# **IAWAKEE – Lach und Sachgeschichten aus Mikroklimaforschung und Informatik**

---

Inhaltsübersicht:

Hintergrund

Einführung Softwaredesign

Einführung Coding-Paradigmen

- Imperative Programmierung

- Objekt-Orientierte-Programmierung

- Deklarative Programmierung

OOP Pattern

Zusammenfassung

- Steigende Bedeutung der Mikroklimaforschung
  - ▶ Große Datensammlungen
- Steigende Bedeutung der Informatik
- KI-Hype
  - ▶ Soziale / Ökologische Kosten

- Aufgabe zur Entwicklung einer Optimierungssoftware für Klima-Maßnahmen
- Verlässliche, hochwertige Software erforderlich

⇒ Softwaredesign von zentraler Bedeutung

|                              |                                                                                                      |
|------------------------------|------------------------------------------------------------------------------------------------------|
| Software-Design              | Das Planen von Software um bestimmte Ziele zu erfüllen                                               |
| Design Patterns <sup>1</sup> | Standard Lösungen für Probleme bei der Softwareentwicklung<br>Werden entdeckt, <u>nicht</u> erfunden |
| Paradigmen                   | Fundamentale Programmierstile                                                                        |

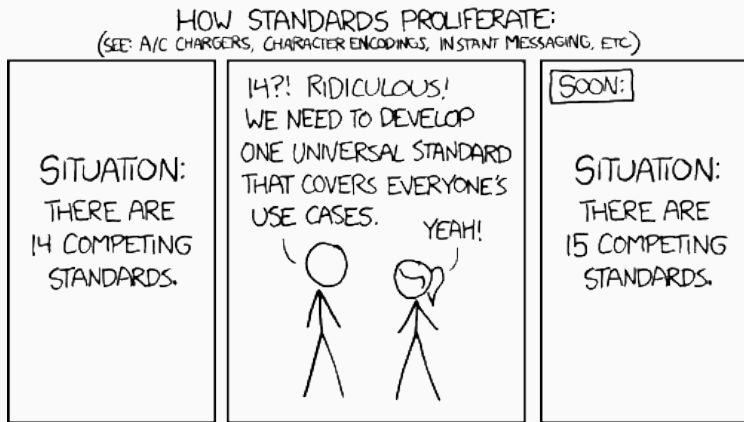
---

<sup>1</sup> „Entwurfsmuster“

## Welche Ziele soll die Software nun erfüllen?

- Testbarkeit
  - ▶ Automatisierbarkeit
- Wartbarkeit
- Erweiterbarkeit
- Kompatibilität

⇒ Standards setzen / Entscheidungen treffen



Unterteilung:

- **Imperative**-Programmierung
  - ▶ **Objekt-Orientierte**-Programmierung (OOP)
- **Deklarative**-Programmierung

Unterscheidung: **State-Frage!**



Was ist State?

- Anwendungszustand
- Die Gesamtheit des Zustands in dem sich ein Programm befindet

State Frage: Wie wird **state** in einer Anwendung gehandhabt?

# Paradigmen – Imperative Programmierung

```
werte = [1,2,3,4,5]
summe = 0
for wert in werte:
    summe = summe + wert
```



## Imperative Programmierung

- Anwendungszustand muss bei jeder Anweisung berücksichtigt werden
- Beschreibt wird **wie** etwas durchgeführt wird

# Paradigmen – Objekt-Orientierte Programmierung

```
class Person
    def __init__(self, name):
        self.name = name
    def greet(self):
        print(self.name + " says hello!")
```

```
ada = Person("Ada Lovelace")
ada.greet()
```

```
alan = Person("Alan Turing")
alan.greet()
```

## Objekt-Orientierte Programmierung

- Anwendungszustand wird in Klassen gekapselt (Datenkapselung)
  - ▶ Kein globaler<sup>1</sup> State; State ist lokal<sup>2</sup>
- Beschreibt **wie** etwas durchgeführt wird

---

<sup>1</sup>global = von überall sichtbar/veränderbar

<sup>2</sup>lokal = nur lokal sichtbar/veränderbar

# Paradigmen Deklarative Programmierung

## Deklarative Programmierung

### Zielsetzung:

- (Veränderlicher) Anwendungszustand existiert nicht.
  - ▶ Setzen wir einmal eine variable dürfen wir sie nicht verändern
  - ▶ State wird in Funktionen „gekapselt“, state darf nicht außerhalb von Funktionen erscheinen

```
a = 100  
def main():  
    a = 1  
    b = 2  
    print(a+b)
```

```
def main():  
    a = 1  
    b = 2  
c = a + b  
    print(c)
```

```
def add(a,b):  
    return a + b  
  
def main():  
    a = 1  
    b = 2  
    print(add(a,b))
```

Ein naiver Versuch:

```
def main():  
    werte = [1,2,3,4,5]  
    summe = 0  
    for wert in werte:  
        summe = summe + wert  
  
main()
```

# Paradigmen – Deklarative Programmierung

Ein naiver Versuch:

```
def main():  
    werte = [1,2,3,4,5]  
    summe = 0  
    for wert in werte:  
        summe = summe + wert
```

main()

```
def main():  
    werte = [1,2,3,4,5]  
    summe = 0  
    for wert in werte:  
        summe = summe + wert
```

main()



# Paradigmen – Deklarative Programmierung

Neue Spielregel:

```
def f(a):  
    return a() + 2  
  
def a():  
    return 1  
  
def main():  
    funktion = a  
    print(funktion()) # schreibt 1  
    print(f(funktion)) # schreibt 3  
    print(f(a))       # schreibt 3
```

Neue Spielregel:

```
def f(a):  
    return a() + 2
```

```
def a():  
    return 1
```

```
def main():  
    funktion = a  
    print(funktion()) # schreibt 1  
    print(f(funktion)) # schreibt 3  
    print(f(a))       # schreibt 3
```

## Ein Designpattern!

Das „Callback-Pattern“:

Eine Funktion kann als Parameter einer anderen Funktion verwendet werden.

## Paradigmen – Deklarative Programmierung

```
def foreach_once(fn, liste, index):  
    fn(liste[index])  
    if index < (len(liste)-1):  
        foreach_once(fn, liste, index + 1)
```

```
def foreach(fn, liste):  
    foreach_once(fn, liste, 0)
```

```
def do_for_all(element):  
    print(element)
```

```
def main():  
    liste = [1,2,3,4,5]  
    foreach(do_for_all, liste)
```

# Paradigmen – Deklarative Programmierung

```
def reduce_once(fn, akk, liste, index):  
    if index < len(liste):  
        newakk = fn(akk, liste[index])  
        res = reduce_once(fn, newakk, liste, index + 1)  
        return res  
    else:  
        return akk
```

*# Reduziert die Liste liste mithilfe der Funktion fn zu einem einzelnen Wert*

```
def reduce(fn, akk, liste):  
    return reduce_once(fn, akk, liste, 0)
```

```
def do_for_all(akk, wert):  
    return akk + wert
```

```
def main():  
    liste = [1,2,3,4,5]  
    akk = 0  
    summe = reduce(do_for_all, akk, liste)  
    print(summe)
```

Klingt unnötig kompliziert?

- Ein einfacheres Beispiel:

```
werte = [1,2,3,4,5]
summe = sum(werte)
print(summe)
```

- Ist äquivalent zu:

```
def do_for_all(akk, wert):
    return akk + wert

werte = [1,2,3,4,5]
akk = 0
summe = reduce(do_for_all, akk, werte)
print(summe)
```

# Paradigmen – Deklarative Programmierung

Ein letztes:

Eine Funktion, die mehrere Parameter nimmt kann wie folgt in eine Serie von Funktionen aufgebrochen werden:

```
def add(a, b):  
    return a + b
```

```
def add_curried(a):  
    def add_b(b):  
        return a + b  
    return add_b
```

```
add5 = add_curried(5) # Erzeugt eine Funktion, die 5 + b rechnet  
print(add5(3))        # Ausgabe: 3 + 5 = 8
```

Diese Praxis heißt **Currying**.

# Pattern in der Objekt-Orientierten Programmierung

---

# Pattern in der Objekt-Orientierten Programmierung

## Factory Method Pattern

```
public interface Postage { String type(); }

public class RegularPostage implements Postage {
    public String type() { return "Regular"; }
}

public class ExpressPostage implements Postage {
    public String type() { return "Express"; }
}

public final class PostageFactory {
    private PostageFactory() {}
    public static Postage by_price(double price_for_product) {
        if (price_for_product > 30) {
            return new ExpressPostage();
        } else {
            return new RegularPostage();
        }
    }
}
```



- Wenn verschiedene Klassen ein Interface implementieren können wir zur Laufzeit entscheiden ein Objekt welcher konkreten Klasse wir zurückgeben
  - ▶ Hier gibt die Methode `by_price` der Klasse `PostageFactory` das interface `Postage` zurück.
  - ▶ Diese Methode heißt `Factory method` oder „Fabrik Methode“

# Pattern in der Objekt-Orientierten Programmierung

## Builder Pattern

```
class House:
    def __init__(self, walls, roof, door):
        self.walls = walls
        self.roof = roof
        self.door = door

class HouseBuilder:
    def __init__(self):
        self._walls = "concrete" # Houses have concrete walls by default
        self._roof = "tile"      # Houses have tile roofs by default
        self._door = "glass"     # Houses have glass by default
    def walls(self, style):
        self._walls = style
        return self
    def roof(self, style):
        self._roof = style
        return self
    def door(self, style):
        self._door = style
        return self
    def build(self):
        return House(self._walls, self._roof, self._door)
```

- Hier Übertragen wir die Verantwortung die Klasse House zu konstruieren an die Klasse HouseBuilder
- Dadurch müssen wir den Konstruktor von House nicht mehr selber aufrufen und können dadurch Standard Werte festlegen.

```
house = HouseBuilder()  
        .walls("brick")  
        .roof("flat")  
        .door("oak")  
        .build()
```

Ein Kompromiss:

- Objektorientierte Programmierung
  - ▶ State sollte in Klassen gekapselt sein
  - ▶ Globalen State vermeiden
- Deklarative Programmierung
  - ▶ State allgemein vermeiden
  - ▶ State-Änderungen vermeiden
  - ▶ Deklarative-Primitives nutzen
  - ▶ Immutability > Datenkapselung
- Design Pattern
  - ▶ Bieten Struktur, Standardisierung und Namenskonvention

# Vielen Dank für Eure Aufmerksamkeit!

Diese Präsentation:

[https://github.com/chaosFaktor/iawakee\\_for\\_dummies/tree/master/output](https://github.com/chaosFaktor/iawakee_for_dummies/tree/master/output)



### Deklarative Programmierung

„Dear Functional Bros” – CodeAesthetic <https://www.youtube.com/watch?v=nuML9SmdbJ4>

### Software Design

CodeAesthetic (Kanal) <https://www.youtube.com/@CodeAesthetic>

### Software Design

„Gang of Four Design Patterns” <https://www.scribd.com/doc/135313382/Gang-of-Four-Design-Patterns-4-0-pdf>

Einfachere Erklärung: <https://refactoring.guru/>



### Informatik Allgemein

The Coding Train <https://www.youtube.com/playlist?list=PLRqwX-V7Uu6Zy51Q-x9tMWIv9cue0FTFA>

CCC Hackerethik <https://www.ccc.de/de/hackerethik>

### Wargames

„CTF: WTF?“ <https://media.ccc.de/v/38c3-ctf-wtf-capture-the-flag-fr-ein>

Over the Wire <https://overthewire.org/wargames/>

Pixelflut <https://c3pixelflut.de/>

### Unterhaltung

Kinder, es tut mir unendlich leid ...

[https://media.ccc.de/v/](https://media.ccc.de/v/gpn21-29-kinder-es-tut-mir-undendlich-leid)

[gpn21-29-kinder-es-tut-mir-undendlich-leid](https://media.ccc.de/v/gpn21-29-kinder-es-tut-mir-undendlich-leid)

„Der Thüring-Test für Wahlsoftware“ – Linus Neumann und Thors-  
ten Schröder

[https://media.ccc.de/v/](https://media.ccc.de/v/38c3-der-thring-test-fr-wahlsoftware)

[38c3-der-thring-test-fr-wahlsoftware](https://media.ccc.de/v/38c3-der-thring-test-fr-wahlsoftware)

„Wir wissen wo dein Auto steht“ –  
Kreil und Flüpke

[https://media.ccc.de/v/](https://media.ccc.de/v/38c3-wir-wissen-wo-dein-auto-steht-volksda)

[38c3-wir-wissen-wo-dein-auto-steht-volksda](https://media.ccc.de/v/38c3-wir-wissen-wo-dein-auto-steht-volksda)

### Unterhaltung

„How to build a submarine and survive” – Nico und Elias

[https://media.ccc.de/v/37c3-11828-how\\_to\\_build\\_a\\_submarine\\_and\\_survive](https://media.ccc.de/v/37c3-11828-how_to_build_a_submarine_and_survive)

Simone Giertz

<https://www.youtube.com/@simonegiertz>

### Unterhaltung

Lilith Wittmann

<https://links.lilithwittmann.de/> bzw  
<https://media.ccc.de/search?p=Lilith+Wittmann>

„Jens Spahns credit score is 'very good'” – Lilith Wittmann

[https://media.ccc.de/v/camp2023-57571-jens\\_spahns\\_credit\\_score\\_is\\_very\\_good](https://media.ccc.de/v/camp2023-57571-jens_spahns_credit_score_is_very_good)

### Unterhaltung

„Heimlich Manöver“ – Arne Sems-  
rott

[https://media.ccc.de/v/  
37c3-11689-heimlich-manover](https://media.ccc.de/v/37c3-11689-heimlich-manover)

Fnord - Nachrichtenrückblick – Fefe und Atoth

[https://media.ccc.de/v/  
38c3-fnord-nachrichtenrckblick-202](https://media.ccc.de/v/38c3-fnord-nachrichtenrckblick-202)

Fefe

<https://blog.fefe.de>

### Unterhaltung

„Illegal Infrastructure: 12 years of hosting in the greyzone” – Kolja und Mark

[https://media.ccc.de/v/](https://media.ccc.de/v/38c3-illegal-infrastructure-12-years-of-hos)

[38c3-illegal-infrastructure-12-years-of-hos](https://media.ccc.de/v/38c3-illegal-infrastructure-12-years-of-hos)

Attack Mining: How to use distributed sensors to identify and take down adversaries (Fortgeschritten)

[https://media.ccc.de/v/](https://media.ccc.de/v/38c3-attack-mining-how-to-use-distributed-s)

[38c3-attack-mining-how-to-use-distributed-s](https://media.ccc.de/v/38c3-attack-mining-how-to-use-distributed-s)

„Alles ist 1 außer der 0” (Film)